

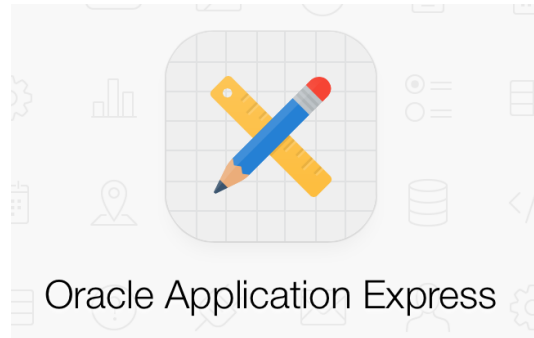
SYSTÈME DE GESTION DE BASES DE DONNÉES

APEX, Blobs et Base64

**3^{ème} Informatique de gestion
2023-2024
Samuel Hiard**

APEX

- APEX = Oracle **AP**plication **E**xpress



- (anciennement Oracle HTML DB)
- Environnement intégré pour un développement web rapide avec Oracle
 - Beaucoup de possibilités via la GUI
 - Intégration code HTML, CSS, Javascript
 - Learning curve (assez) importante
- API callable via une procédure PL/SQL
 - Ce qui nous intéresse ici

Package HTB

- Ecrit par mon prédécesseur
 - Merci Mr DeDijcker
- Ensemble de procédures « wrapper » pour faciliter l'envoi de données
 - `procedure append(line varchar2);`
 - `procedure append_nl(line varchar2);`
 - `procedure reset;`
 - `procedure send(c clob,content_type varchar2 default 'text/html',status_code integer default 200);`
 - `procedure send(v varchar2,content_type varchar2 default 'text/html',status_code integer default 200);`
 - `procedure send(content_type varchar2 default 'text/html',status_code integer default 200);`

Etape 1 : Création d'une procédure

create or replace PROCEDURE testPOST(reqBody clob) IS

```
    city_param locations.city%type;
    state_param locations.state_province%type;
    c sys_refcursor;
BEGIN
    -- On parse le JSON reçu en paramètre
    apex_json.parse(reqBody);

    -- Récupération de chaque champ
    --apex_json.get_number('<valeur>'); si paramètre numérique
    city_param := apex_json.get_varchar2('city');
    state_param := apex_json.get_varchar2('state');

    OPEN c
    FOR SELECT location_id, street_address, postal_code, city, state_province,
country_id
    FROM Locations
    WHERE UPPER(city) LIKE '%' || UPPER(city_param) || '%'
    AND UPPER(state_province) LIKE '%' || UPPER(state_param) || '%' ;
```

Etape 1 (suite)

```
apex_json.initialize_clob_output;  
apex_json.write(c);
```

```
htb.send(apex_json.get_clob_output, 'application/json; charset=utf-8');  
apex_json.free_output;
```

EXCEPTION

when others then

```
htb.reset;  
htb.append_nl('<html><body>');  
htb.append_nl('<p>SQLERRM : ' || htf.escape_sc(sqlerrm) || '</p>');  
htb.append_nl('</body></html>');  
htb.send(status_code => 500);
```

END testPOST;

Etape 2 : Ecrire un gestionnaire POST

```
begin  
    testPOST(:body_text);  
end;
```

Etape 3 : Test dans POSTMAN

POST ⌵ http://localhost:8080/ords/hr/demo/locations Send ⌵

Params Authorization Headers (8) **Body** ● Pre-request Script Tests Settings Cookies

● none ● form-data ● x-www-form-urlencoded ● raw ● binary ● GraphQL **JSON** ⌵ Beautify

```
1 {
2   "city": "o",
3   "state": "o"
4 }
```

Body Cookies Headers (3) Test Results Status: 200 OK Time: 215 ms Size: 1.23 KB Save Response ⌵

Pretty Raw Preview Visualize JSON ⌵ ⌵

```
1 {
2   {
3     "LOCATION_ID": 1200,
4     "STREET_ADDRESS": "2017 Shinjuku-ku",
5     "POSTAL_CODE": "1689",
6     "CITY": "Tokyo",
7     "STATE_PROVINCE": "Tokyo Prefecture",
8     "COUNTRY_ID": "JP"
9   },
10  {
11    "LOCATION ID": 1500,
```

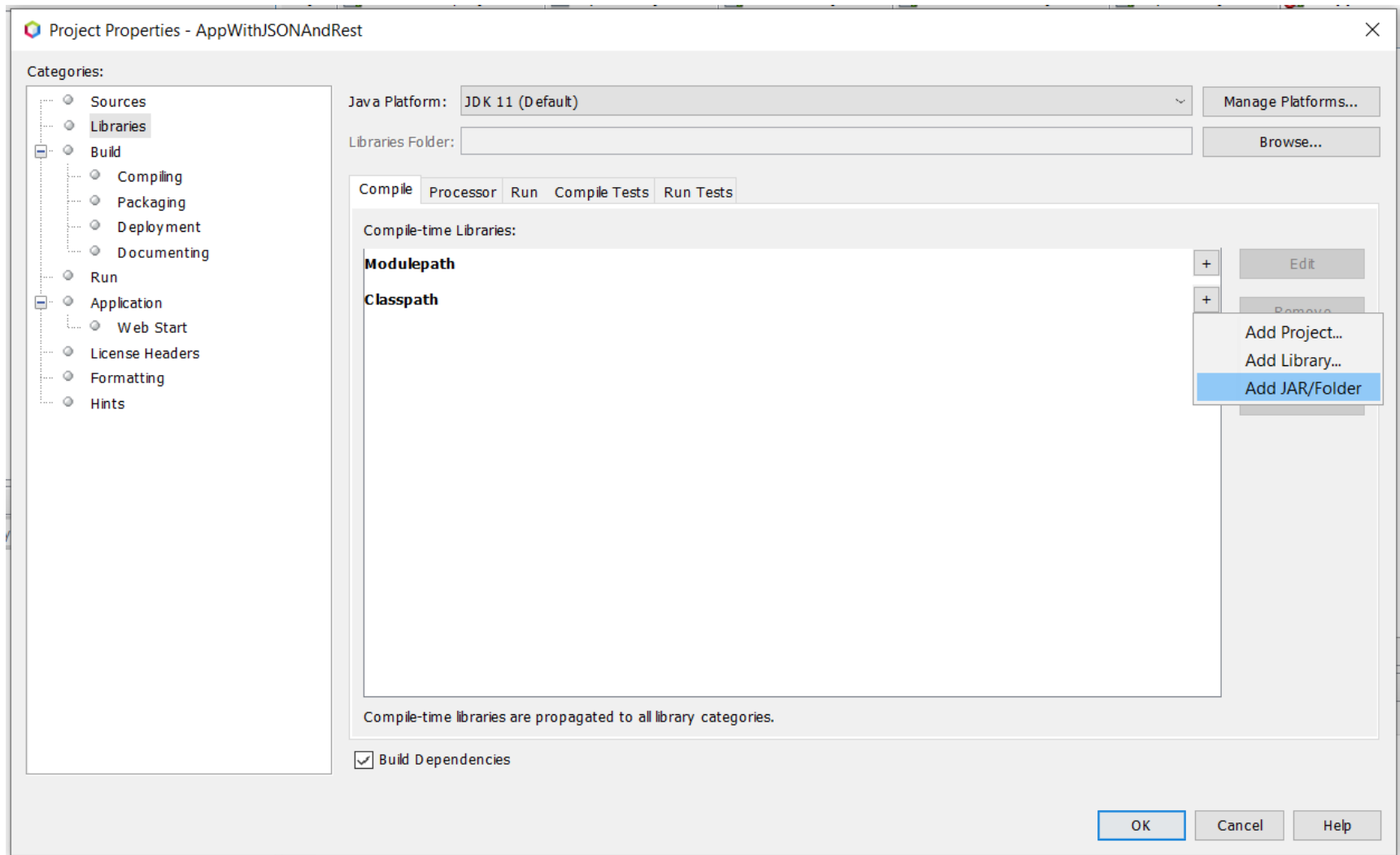
🏠 🏃 🗑️ 🔍

Package org.JSON

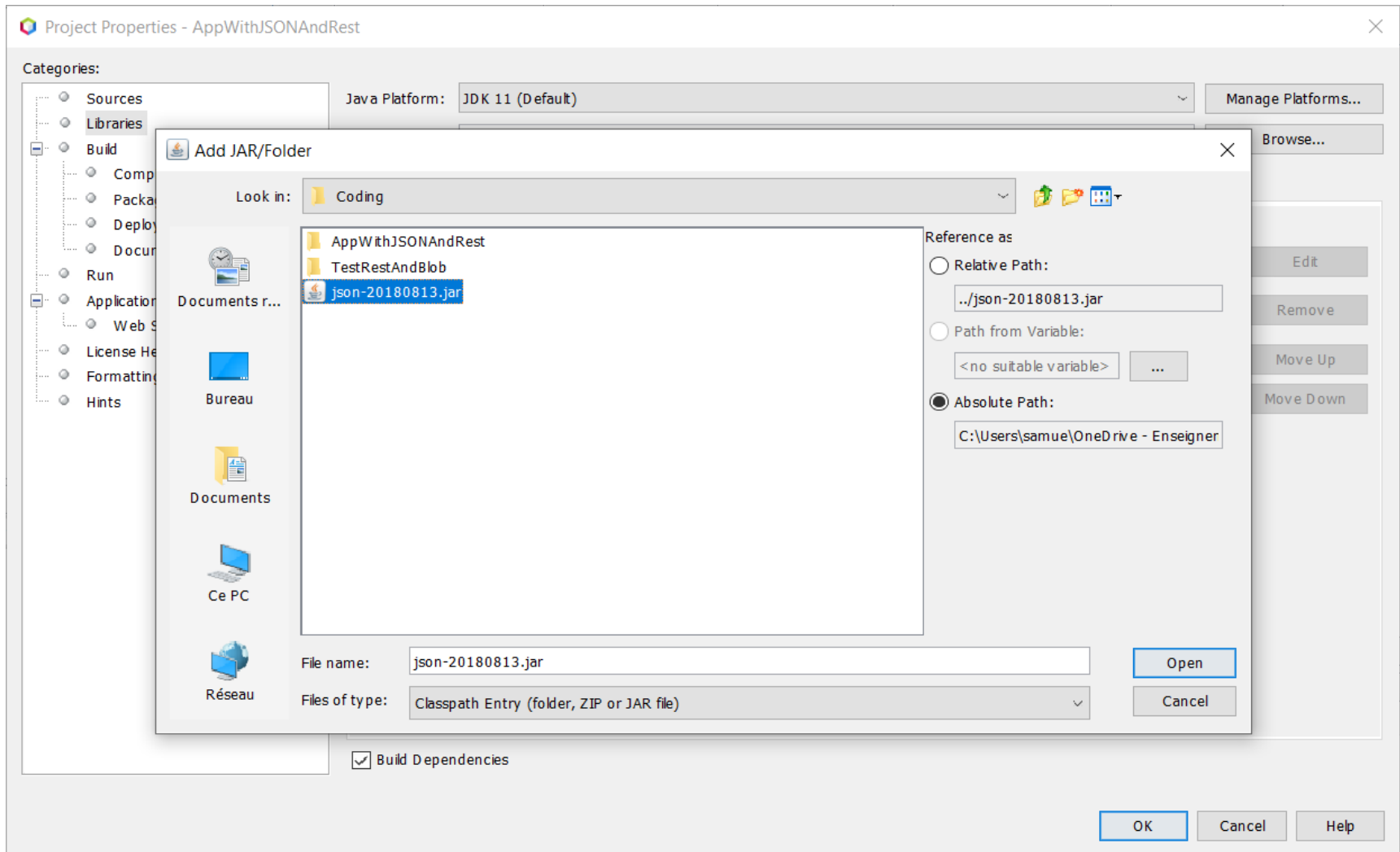
- Permet de manipuler le format JSON plus facilement

- Opérations pour
 - Construire un élément JSON
 - A mettre dans le contenu de la requête POST
 - Parser un document JSON
 - Récupération d'une valeur en fonction d'une clé
 - Parcourt des valeurs définies
 - Gestion des objets et des tableaux

Ajouter le package JSON dans le classpath



Ajouter le package JSON dans le classpath (2)



Appel vers une page WEB

- Connection vers la page
 - `URL url = new URL("url_du_site");`
 - `URLConnection con = (URLConnection) url.openConnection();`
- Choix du verbe
 - `con.setRequestMethod("GET");`
- Envoi des données si POST
 - Voir exemple plus loin
- Récupération de la réponse dans un `bufferedReader`

Exemple GET

```
String line;
StringBuilder result = new StringBuilder();
int id = 100;
try {
    URL url = new URL("http://localhost:8080/ords/hr/demo/employees/"+id);
    HttpURLConnection con = (HttpURLConnection) url.openConnection();

    con.setRequestMethod("GET");

    BufferedReader rd = new BufferedReader(new InputStreamReader(con.getInputStream()));

    while ((line = rd.readLine()) != null)
    {
        result.append(line);
        System.out.println(line);
    }
    rd.close();

} catch (Exception e) {
    System.out.println("Error in getting details : " + e);
}
```

Passer des paramètres à POST

■ Avec un JSONObject

```
import org.json.*;
```

```
(...)
```

```
JSONObject obj = new JSONObject();
```

```
obj.put("city", "o");
```

```
obj.put("state", "o");
```

```
String urlParameters = obj.toString();
```

```
byte[] postData =  
urlParameters.getBytes(StandardCharsets.UTF_8 );
```

```
int postDataLength = postData.length;
```

Connection POST et envoi de paramètres

```
URL url = new URL("http://localhost:8080/ords/hr/demo/locations");
URLConnection conn = (URLConnection) url.openConnection();

conn.setDoOutput(true);
conn.setInstanceFollowRedirects(false);
conn.setRequestMethod("POST"); //Verbe POST
conn.setRequestProperty("Content-Type", "application/json");
conn.setRequestProperty("charset", "utf-8");
conn.setRequestProperty("Content-Length",
    Integer.toString(postDataLength));
conn.setUseCaches(false);

DataOutputStream wr = new DataOutputStream(conn.getOutputStream());
wr.write(postData);
```

Parsing des résultats

- Dans org.JSON
 - JSONObject
 - JSONArray
- Création d'un de ces deux objets à partir d'une chaîne de caractère
 - (représentant l'objet JSON)
- Méthodes utiles
 - (JSONObject/JSONArray).getJSONObject(key/pos)
 - (JSONObject/JSONArray).getJSONArray(key/pos)
 - (JSONArray).get(pos)
 - (JSONObject).keys()
 - (JSONObject).get(key)
 - (JSONObject).has(key)
 - ...

Exemple

```
//Parsing dans le JSON reçu
JSONArray received = new JSONArray(new String(result2));
for(int i = 0; i < received.length(); i++)
{
    JSONObject currentObject = received.getJSONObject(i);
    Iterator it = currentObject.keys();
    while(it.hasNext())
    {
        String key = (String)it.next();
        System.out.println(key + " : " +
            currentObject.get(key));
    }
}
```

Les LOB

- Large OBject
- Type de donnée SQL qui permet de stocker des données binaires dans la DB Oracle
- 4 types de LOB (Max. 4Gb) :
 - BLOB : Contenu binaire
 - CLOB : Contenu texte
 - Rappel : Varchar2 contient max 32,767 bytes
 - NCLOB : Idem mais seulement unicode
 - BFILE : Pointeur vers un blob en-dehors de la BD

Le problème des blobs

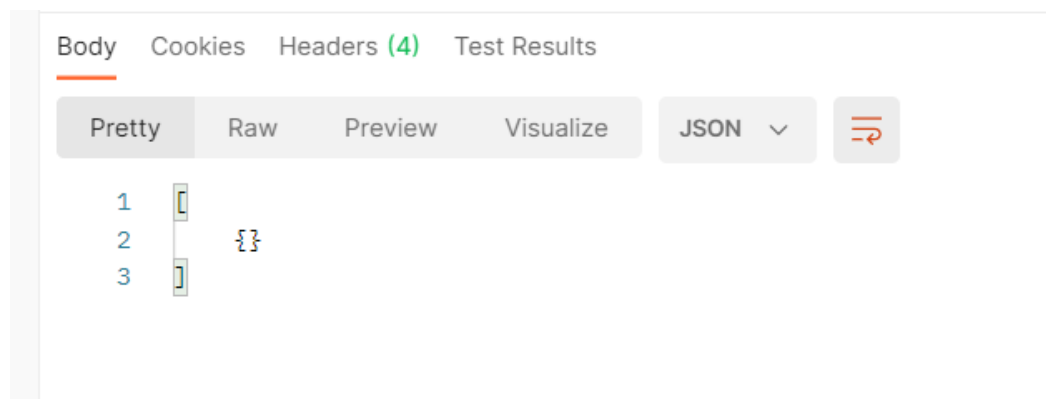
- Les Blobs sont des objets binaires
- Ils ne peuvent pas transiter tels quels sur le réseau
- Le gestionnaire ORDS va automatiquement transformer l'objet BLOB en chaîne de caractères grâce à l'encodage Base64
 - Sauf si vous passez par un code PL/SQL...

Le problème des blobs

- Les Blobs sont des objets binaires
- Ils ne peuvent pas transiter tels quels sur le réseau
- Dans le gestionnaire ORDS :

```
OPEN c
FOR SELECT img
FROM BlobTest
FETCH NEXT 1 ROW ONLY ;
```

- Résultat sur PostMan :



Encodage en base 64

- Transformer les objets binaires en chaînes de caractère
- Plusieurs méthodes
 - Quoted-printable encoding
 - Replace les caractères « bizarres » par un code précédé de =
 - Ex : caractère → `caract=E8re`
 - Plutôt pour les chaînes en grande partie valides
 - Base 64 encoding
 - Considère les données comme une chaîne de bits
 - Encode individuellement chaque groupe de 6 bits
 - 64 valeurs possibles (d'où le nom)
 - Chaque valeur a une correspondance vers un caractère dans une table.

Implémentation

- Pas besoin de réinventer la roue :

```
create or replace FUNCTION base64encode(p_blob IN BLOB) RETURN CLOB
-- -----
-- File Name      : https://oracle-base.com/dba/miscellaneous/base64encode.sql
-- Author         : Tim Hall
-- Description    : Encodes a BLOB into a Base64 CLOB.
-- Last Modified: 10/11/2021 by Samuel HIARD to remove the newlines
-- -----
IS
  l_clob CLOB;
  l_step PLS_INTEGER := 12000; -- make sure you set a multiple of 3 not higher than 24573
BEGIN
  IF(p_blob is not null) THEN
    FOR i IN 0 .. TRUNC((DBMS_LOB.getlength(p_blob) - 1)/l_step) LOOP
      l_clob := l_clob ||

      replace(
        replace(
          utl_raw.cast_to_varchar2(utl_encode.base64_encode(DBMS_LOB.substr(p_blob, l_step, i * l_step + 1))),
          chr(10)),
          chr(13)
        );
    END LOOP;
  END IF;
  RETURN l_clob;
END base64encode;
```

Utilisation de la fonction

■ Dans le handler

```
OPEN c
FOR SELECT base64encode(img)
FROM BlobTest
FETCH NEXT 1 ROW ONLY ;
```

■ Résultat

Body Cookies Headers (4) Test Results

Status: 200 OK Time: 805 ms Size: 560.03 KB Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "BASE64ENCODE(IMG)": "iVBORw0KGgoAAAANSU...
3   +C8SBAECPOLI5exid+8wezM7ukVVV3dVl66szEoZGRnK3Yw/mJm7uYqIFNUza91ZEeFu4pnZs/e+75kS41hpKoLWlY+vNNSVobXwf9m/QhAKtABE+V1F3kII
4   +7yUUS5uXTqfnjLdJo6U+bhaq1ye5l0U0tWVXRK9ExPn9Vdp7+llAghvD8otrNpt/9XzMv155fll6mUQmuBUoo4ju1vXXhvytcaS4sgCAJAKySK0ThhNI4Zj7P05g
5   +UUitiJVGufyU6eu0TWxcWwxKq3K/aGFo80rv/vJlFtoJAJl7LoTI4lawsN+Prg2DQ0baz/VLGAWEYUCrEdFsNmg0IsIgSLmnjn/
6   +9MrJ2JfcsI0zkiBQCwLASBoIwDGg0IgiP0FoRj8q0udePAFJKU1Wli0PEXLNjWSMQgJSGlQVMf2RjWIPQ+X5NXN+BSGWczvhV2LxwfZ3Rk7ajNHUxbStzveb4w/Svq37V
7   +PI6TyiEkAiCtL2VUqUUVePALzdrf18ulfsu41GRY6FpsqKKD7Q2bZc2ewW9AFIISGUcoHSBffPy0SsqH88vV2ssBwjy4y7LMethTKcJMqbVKs+qNmL6yNHsXmqN
8   +U9MaCuN0B6dNp0rUx0Yd1AIoQ1JQoJ241QzVjHPX7zm7/7mc7767Dk/+vR9/rP//FM2tucRgVeSVl6/1
9   +m9at3iZEGWT33fCwGBFEhpZLKUAPu0EhWD1lbuSwTCfEoQQpFowXgk+Ku/+jkfffQ+17ZwcvxiWkfk27+CftK49TrVjfU6XwbUZnHc
10  +Kk1SMdD7mFRlwpEIw1l1u2m0MHUrvnbywecR07vIiwDD3CnfFHU0tjoed6Zhki6vDSLncEL0uc8q+V/I0MZz/
```

Décodage

- Objectif : pouvoir récupérer le contenu binaire
- Traitement
 - Ex : Transformation de ce contenu dans une image et association à un JLabel
- En java, classe Base64 pour décoder
 - Pas demandé dans le projet

Côté Java



Et dans l'autre sens?

- On a vu comment récupérer une image depuis la base de données, comment envoyer une image à la base de données?
- Principe inverse :
 - Java encode l'image en base64
 - L'image est envoyée dans un JSON à un gestionnaire POST
 - Oracle récupère le contenu, le transforme en blob, et l'insère dans sa base de données

Côté java

```
File fi = new File("C:\\HEPL\\petitchat.jpg");
byte[] byteArray = Files.readAllBytes(fi.toPath());
String encoded = Base64.getEncoder().encodeToString(byteArray);

JSONObject json = new JSONObject();
json.put("image", encoded);
String urlParameters = json.toString();
byte[] postData = urlParameters.getBytes(StandardCharsets.UTF_8 );
int postDataLength = postData.length;

URL url = new URL("http://localhost:8080/ords/hr/demo/blob");
URLConnection conn = (URLConnection) url.openConnection();
conn.setDoOutput(true);
conn.setInstanceFollowRedirects(false);
conn.setRequestMethod("POST"); //Verbe POST
conn.setRequestProperty("Content-Type", "application/json");
conn.setRequestProperty("charset", "utf-8");
conn.setRequestProperty("Content-Length", Integer.toString(postDataLength));
conn.setUseCaches(false);

DataOutputStream wr = new DataOutputStream(conn.getOutputStream());
wr.write(postData);

BufferedReader rd = new BufferedReader(new InputStreamReader(conn.getInputStream()));
String line;
while ((line = rd.readLine()) != null) {
    System.out.println(line);
}
rd.close();
```

Côté bases de données

- Gestionnaire POST :

```
BEGIN
```

```
    handleordspost(:body_text);
```

```
    http.prn('Image posted');
```

```
END;
```

La fonction de gestion du handler

```
create or replace PROCEDURE HandleORDSPOST(reqBody clob) IS  
    myimage clob;  
BEGIN
```

```
-- On parse le JSON reçu en paramètre  
    apex_json.parse(reqBody);
```

```
-- Récupération de chaque champ  
--apex_json.get_number('<valeur>'); si paramètre numérique  
    myimage := apex_json.get_clob('image');
```

```
    INSERT INTO blobtable(image)  
VALUES(base64decodeclobtoblob(myimage));  
    COMMIT;  
END;
```

La fonction de conversion

```
create or replace FUNCTION base64decodetoblob(p_clob CLOB) RETURN BLOB
-----
-- File Name      : https://oracle-base.com/dba/miscellaneous/base64decode.sql
-- Author         : Tim Hall
-- Description    : Decodes a Base64 CLOB into a BLOB
-- Last Modified: 09/11/2011
-----
IS
  l_blob      BLOB;
  l_raw       RAW(32767);
  l_amt       NUMBER := 7700;
  l_offset    NUMBER := 1;
  l_temp      VARCHAR2(10000 char);
BEGIN
  BEGIN
    DBMS_LOB.createtemporary (l_blob, FALSE, DBMS_LOB.CALL);
    LOOP
      DBMS_LOB.read(p_clob, l_amt, l_offset, l_temp);
      l_offset := l_offset + l_amt;
      l_raw    := UTL_ENCODE.base64_decode(UTL_RAW.cast_to_raw(l_temp));
      DBMS_LOB.append (l_blob, TO_BLOB(l_raw));
    END LOOP;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      NULL;
  END;
  RETURN l_blob;
END;
```